

C++ - разширени отговори за изпит

Само C++ и общо ООП. Всеки въпрос е разширен с повече обяснение, а където е полезно има кратък примерен код.

1. Обекти и класове. Дефиниция на клас. Общи понятия и концепции.

Класът е потребителски тип, който обединява данни и функции за работа с тях. Обектът е конкретна инстанция на класа и има собствено състояние. Основните ООП концепции са капсулация, наследяване, полиморфизъм и абстракция. Капсулацията скрива вътрешната реализация, а абстракцията показва само важната част от поведението.

```
class User {
private:
    std::string name;
public:
    User(std::string n) : name(n) {}
    std::string getName() const { return name; }
};

User u("Ivan");
```

2. Методи и параметри. Даннови членове и пропъртита.

Методът е функция, която принадлежи на клас и описва поведението на неговите обекти. Параметрите подават входни данни към метода, а данновите членове пазят състоянието на обекта. В C++ няма вградени properties като в C#, затова обикновено се използват getter и setter методи. Това позволява валидация и контрол върху промяната на данните.

```
class Account {
private:
    double balance = 0;
public:
    double getBalance() const { return balance; }
    void setBalance(double b) {
        if (b >= 0) balance = b;
    }
};
```

3. Модификатори на достъп в клас.

public членовете са достъпни от външен код, private - само от самия клас и приятелите му, а protected - и от наследниците. Те се използват за капсулация и ограничаване на директния достъп до вътрешното състояние. В class достъпът по подразбиране е private, а в struct е public.

```
class Base {
private:
    int a;
protected:
    int b;
public:
    int c;
};
```

4. Accessor-методи. Mutator-метод.

Accessor или getter методът връща стойност от обекта, без по принцип да го променя. Mutator или setter методът променя вътрешното състояние. Чрез тези методи можем да проверяваме дали новата стойност е валидна и да пазим инвариантите на класа.

```
class Person {
private:
    int age = 0;
public:
    int getAge() const { return age; }
    void setAge(int value) {
        if (value >= 0) age = value;
    }
};
```

5. Методи на клас. Видове и модификатори. Припокриване.

Методите могат да бъдат обикновени, const, static, virtual и други. const метод не трябва да променя логическото състояние на обекта, а static метод принадлежи на класа, не на конкретен обект. Overloading означава няколко функции с едно име, но различни параметри. Компиляторът избира правилната версия според извикването.

```
class Printer {
public:
    void print(int x) {}
    void print(const std::string& s) {}
};
```

6. Програмни практики: цифров часовник - диаграми на класове и обекти.

Цифров часовник може да бъде разделен на класове с ясни отговорности, например ClockDisplay и NumberDisplay. Класовата диаграма показва типовете и връзките между тях, а обектната диаграма - конкретни инстанции в даден момент. Този подход намалява сложността и позволява отделните части да се променят независимо.

7. Групиране на обекти. Колекции и итератори.

Колекциите съхраняват множество обекти, например vector, list, map и set. Итераторът предоставя общ начин за обхождане на елементите на контейнера, без кодът да зависи от вътрешната му реализация. В практиката често се използва range-based for, който вътрешно работи чрез итератори.

```
std::vector<int> nums{1, 2, 3};

for (auto it = nums.begin(); it != nums.end(); ++it) {
    std::cout << *it << " ";
}
```

8. Проектиране на класова йерархия в обектно-структурирана програма.

Класовата йерархия поставя общите характеристики в базов клас, а специализираните - в производни класове. Наследяване е подходящо, когато между типовете има истинска връзка is-a. Добрата йерархия позволява полиморфизъм и намалява дублирането, но не трябва да се използва само за повторна употреба на код.

```
class Animal {
public:
    virtual void sound() = 0;
    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    void sound() override { std::cout << "Woof"; }
};
```

9. Оценка на качеството на кода. Свързаност и структурираност.

Качественият код е четим, лесен за тестове, промяна и повторна употреба. Ниска свързаност означава класовете да имат минимални зависимости помежду си. Висока кохезия означава един клас да съдържа логически свързани отговорности. Така промените остават локални и рискът от странични ефекти е по-малък.

10. Качество на код: дублиращи се фрагменти. Целево-ориентиран проект.

Дублираният код затруднява поддръжката, защото една и съща логика трябва да се променя на няколко места. Общото поведение се извлича във функция, клас или друга подходяща абстракция. Целта не е да се премахне всяко повторение на всяка цена, а да се избегне дублиране на една и съща бизнес логика.

11. Проектиране на обектно-структуриран код.

При проектирането се определят основните обекти, техните отговорности, данни и взаимодействия. Всеки клас трябва да има ясна роля и малък публичен интерфейс. Детайлите на реализацията се скриват, а зависимостите между класовете се поддържат минимални. Често композицията е по-гъвкава от наследяването.

12. Класове и обекти: разделяне на декларация и дефиниция. Създаване и унищожаване на обекти.

В C++ декларацията на класа обикновено се поставя в .h/.hpp, а дефинициите на методите - в .cpp. Това намалява зависимостите и подобрява организацията на проекта. Обектът се създава чрез конструктор и при края на живота му се извиква деструктор.

```
// User.h
class User {
public:
    void hello();
};

// User.cpp
void User::hello() {
    std::cout << "Hello";
}
```

13. Конструктори и деструктори. Видове конструктори.

Конструкторът инициализира нов обект, а деструкторът освобождава ресурсите му. В C++ има default, параметризиран, сору и move конструктор. Конструкторите не могат да бъдат virtual или static. При полиморфен базов клас деструкторът обикновено трябва да бъде virtual.

```
class File {
public:
    File() {}
    File(const File& other) {}
    File(File&& other) noexcept {}
    virtual ~File() = default;
};
```

14. Дефиниране на връзки. Взаимодействия на обекти по вертикала и хоризонтала.

Вертикалните връзки обикновено описват наследяване между базов и производен клас. Хоризонталните връзки описват взаимодействие чрез асоциация, агрегация или композиция. Обектите трябва да комуникират чрез публични методи, а не чрез директен достъп до вътрешните си данни.

15. Подтипове, подкласове и присвоявания. Предаване на параметри.

Подкласът наследява базов клас и може да бъде използван като негов подтип, ако спазва договора му. При копиране на производен обект в базов обект по стойност може да възникне object slicing. За полиморфизъм е правилно да използваме указател или референция към базовия тип.

```
Dog d;
Animal& a = d;    // няма slicing
a.sound();        // извиква Dog::sound()
```

16. Наследяемост. Полиморфизъм. Достъп до методи и данни.

Наследяването позволява производният клас да използва и разширява поведението на базовия. Полиморфизмът позволява една и съща операция да има различна реализация според действителния тип на обекта. Достъпът до членовете зависи от public, protected и private, както и от вида наследяване.

17. Виртуализация и реализация на полиморфизма. Абстрактни класове и виртуални функции.

virtual функцията позволява динамично избиране на реализацията при работа през базова референция или указател. Абстрактният клас има поне една pure virtual функция и не може да бъде инстанциран директно. Производният клас реализира функцията чрез override, а базова реализация може изрично да се извика чрез Base::method().

```
class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

class Square : public Shape {
    double side;
public:
```

```

    Square(double s) : side(s) {}
    double area() const override { return side * side; }
};

```

18. Вграждане на обекти. Сору-конструктори. Присвоявания и обекти.

Композицията означава един обект да съдържа друг като свой член - връзка has-a. Сору конструкторът създава нов обект от съществуващ, а операторът за присвояване променя вече съществуващ обект. При класове, които управляват ресурси, трябва да се спазват Rule of 3/5 или още по-добре Rule of 0.

```

class A {
public:
    A(const A& other) { /* копиране */ }

    A& operator=(const A& other) {
        if (this != &other) {
            // копиране
        }
        return *this;
    }
};

```

19. Референтни параметри. Работа с референции. Връщане на референции.

Референцията е псевдоним към съществуващ обект и позволява работа без копиране. const T& е подходящо за четене на големи обекти, а T& позволява промяна на оригинала. Не трябва да се връща референция към локална променлива, защото тя се унищожава при излизане от функцията.

```

void increment(int& x) {
    ++x;
}

void print(const std::string& s) {
    std::cout << s;
}

```

20. Конструиране на вградени обекти. Деструкция на вградени обекти.

Вградените обекти се конструират преди тялото на конструктора на съдържащия клас. Те се инициализират чрез member initializer list и по реда, в който са декларирани в класа. При унищожаване редът е обратен.

```

class Engine {};

class Car {
    Engine engine;
public:
    Car() : engine() {}
};

```

21. Заделяне на обекти от динамичната памет.

Динамичните обекти традиционно се създават с new и освобождават с delete. Неправилното управление може да доведе до memory leak, dangling pointer, double delete и неясна собственост. Модерният C++ използва RAII и smart pointers като unique_ptr и shared_ptr, за да управлява автоматично живота на ресурсите.

```

auto p = std::make_unique<User>("Ivan");
// delete не е нужен - паметта се освобождава автоматично

```

22. Приятелски класове и приятелски функции. Статични членове на клас.

friend функцията или клас получава достъп до private и protected членовете, без да е обикновен член на класа. Това създава силна зависимост и трябва да се използва ограничено. static членът принадлежи на класа като цяло и е общ за всички негови обекти.

```

class Counter {
public:
    static int count;
    Counter() { ++count; }
};

int Counter::count = 0;

```

23. Припокриване на оператори. Същност, ограничения.

Operator overloading позволява стандартни оператори като +, == и << да работят с потребителски типове. Не могат да се създават нови оператори или да се променят приоритетът, асоциативността и броят на операндите. Добрата реализация трябва да пази естествения смисъл на оператора.

```
struct Point {
    int x, y;

    Point operator+(const Point& other) const {
        return {x + other.x, y + other.y};
    }
};
```

24. Преобразувания и операции-преобразувания.

C++ поддържа неявни и явни преобразувания. За явни преобразувания се използват static_cast, dynamic_cast, const_cast и reinterpret_cast, всеки с различно предназначение. Потребителските типове могат да дефинират conversion operators и конструктори за преобразуване.

```
double d = 3.7;
int x = static_cast<int>(d);

class Number {
public:
    operator int() const { return 42; }
};
```

25. CASE: обектно-структурирана програмна система - плащане.

Системата за плащане може да има обща абстракция Payment и конкретни реализации като CardPayment и BankPayment. UML диаграмата описва класовете, методите, данните и връзките между тях. Полиморфизмът позволява различните плащания да се обработват през общ интерфейс, без кодът да знае конкретния тип.

```
class Payment {
public:
    virtual void pay(double amount) = 0;
    virtual ~Payment() = default;
};

class CardPayment : public Payment {
public:
    void pay(double amount) override {
        std::cout << "Card payment: " << amount;
    }
};
```

26. Разширяване функционалността на системата за плащания: интерфейси.

В C++ интерфейс обикновено се реализира като абстрактен клас с pure virtual функции и virtual деструктор. Конкретните класове наследяват интерфейса и реализират неговия договор. Така може да се добавя нов начин на плащане, без да се променя кодът, който работи през интерфейса.

28. Софтуерни контракти. Пред- и пост-условия. Инварианти.

Предусловието определя какво трябва да бъде вярно преди извикване на операцията. Постусловието описва какво се гарантира след успешно изпълнение, а инвариантът е условие, което трябва да остава валидно за обекта. Контрактите правят очакванията между клиента и реализацията ясни.

```
double withdraw(double amount) {
    // предусловие: amount > 0 и amount <= balance
    // постусловие: balance е намален с amount
}
```

30. Контракт и наследяване. Ограничаване на областта в дъщерен обект.

Производният тип трябва да спазва договора на базовия, за да може да го замества коректно. Той не трябва да изисква по-силни предусловия или да предоставя по-слаби постусловия. Ако наследникът приема по-малко валидни случаи от базовия тип, полиморфният код може да се счупи.

31. Обектен дизайн: SOLID, Open/Closed и Single Responsibility.

SOLID е набор от принципи за по-модулен и поддържаем обектен дизайн. Open/Closed изисква класовете да са отворени за разширяване, но затворени за ненужна промяна. Single Responsibility означава един клас да има една основна отговорност и една логическа причина за промяна.

32. Обектен дизайн: принцип на верижната отговорност.

Chain of Responsibility организира обработчиците във верига. Всеки обработчик решава дали да обработи заявката или да я предаде към следващия. Така изпращачът не зависи директно от конкретен получател и веригата може лесно да се разширява.

```
class Handler {
public:
    virtual bool handle(int request) = 0;
    virtual ~Handler() = default;
};
```

33. Обектен дизайн: double dispatch в run-time.

Double dispatch позволява изборът на операция да зависи от динамичния тип на два обекта. В C++ това често се реализира чрез виртуални функции и Visitor pattern. Подходът е полезен, когато поведението зависи едновременно от типа на посетителя и посещения обект.

37. Lambda изрази. Функции-обекти и функции през указатели.

Lambda е анонимна callable функция, дефинирана директно на мястото на употреба. Тя може да прихваща стойности от външния scope и често заменя малки functor класове или function pointers. Ламбдите са много удобни при STL алгоритми, callbacks и асинхронни задачи.

```
std::vector<int> nums{3, 1, 2};

std::sort(nums.begin(), nums.end(),
    [](int a, int b) { return a < b; });
```

38. Синтактични елементи на lambda изразите в C++.

Основният синтаксис е [capture](parameters) -> return_type { body }, като return type често може да се пропусне. [=] прихваща използваните външни променливи по стойност, а [&] - по референция. Може да се прихваща и конкретна променлива, например [x] или [&x;].

```
int n = 10;

auto add = [n](int x) {
    return x + n;
};

std::cout << add(5); // 15
```

39. Lambda изрази в асинхронни изчисления. Определяне на типа.

Lambda може директно да се подаде към std::async или std::thread като задача. Всяка lambda има уникален closure тип, генериран от компилатора, затова обикновено използваме auto. При capture по референция трябва да внимаваме прихванатият обект да съществува, докато асинхронната задача работи.

```
auto future = std::async(std::launch::async, [] {
    return 42;
});

std::cout << future.get();
```

40. Стандартизирани средства в C++ за паралелизация.

C++ предоставя std::thread, std::async, std::future, mutex-и и други примитиви за конкурентно изпълнение. std::thread представя нишка и трябва коректно да бъде join-ната или управлявана. std::async стартира задача и връща future, чрез който можем да получим резултата.

```
std::thread t([] {
    std::cout << "Hello from thread";
});

t.join();
```

41. Синхронизация при паралелни изчисления. Атомарни типове.

Синхронизацията предотвратява data races при едновременен достъп до общо състояние. `std::atomic` предоставя атомарни операции за подходящи типове без обикновен mutex. Типичен пример е брояч, който се увеличава безопасно от няколко нишки.

```
std::atomic<int> counter{0};

counter.fetch_add(1);
```

42. Взаимни изключвания при паралелни изчисления и примитиви.

`std::mutex` гарантира, че само една нишка може да изпълнява защитен критичен участък в даден момент. `std::lock_guard` и `std::unique_lock` управляват заключването чрез RAII. Това намалява риска mutex да остане заключен при return или exception.

```
std::mutex m;

void work() {
    std::lock_guard<std::mutex> lock(m);
    // критична секция
}
```

43. Условни променливи при паралелни изчисления.

`std::condition_variable` позволява нишка ефективно да чака условие, без да върти празен цикъл. Чакащата нишка освобождава mutex-а и се събужда след `notify_one()` или `notify_all()`. Типичен пример е producer-consumer моделът.

```
std::condition_variable cv;
std::mutex m;
bool ready = false;

std::unique_lock<std::mutex> lock(m);
cv.wait(lock, [] { return ready; });
```

48. Обекти в паметта - особености и програмни грешки.

Обектите могат да имат автоматичен, динамичен или друг вид живот според начина на създаване. Типични грешки са out-of-bounds достъп, dangling pointers, use-after-free, неинициализирани указатели и memory leaks. RAII, стандартните контейнери и smart pointers значително намаляват риска от тези проблеми.

49. Управление на памет в Linux системи: служебни структури.

Адресното пространство на процеса обикновено включва области като code/text, data, heap и stack. Динамичната памет се управлява от allocator върху heap-а чрез механизми като malloc/free или C++ new/delete. В самия конспект се споменава `unlink()`, но той не е стандартен C++ механизъм за освобождаване на обекти и за точния изпитен смисъл трябва да се следва лекцията.

50. Препълване на буфер. Поглед отвътре.

Buffer overflow възниква, когато програмата запише повече данни, отколкото буферът може да побере. Това може да повреди съседна памет, да причини срив или да промени контролни данни на програмата. Основната защита е проверка на границите и използване на по-безопасни C++ абстракции и контейнери.

```
char buffer[8];
// Опасно е да се записват повече от 8 байта в buffer.
// По-безопасно: std::string или std::array с проверки.
```

51. Атака от тип „frontlink“.

В конспекта терминът е даден като отделна тема, но липсват дефиниция и пример, които да позволят надеждно обяснение само от предоставения материал. Затова за този въпрос не е коректно да се измисля съдържание. За изпитния отговор трябва да се използва конкретната дефиниция от лекциите на преподавателя.

52. Double-free vulnerabilities.

Double free възниква, когато един и същ динамично заделен ресурс бъде освободен повече от веднъж. Това води до undefined behavior и може да повреди вътрешните структури на allocator-а. RAII и unique_ptr предотвратяват проблема чрез ясна и автоматична собственост върху ресурса.

```
auto p = std::make_unique<int>(5);  
// Паметта се освобождава точно веднъж автоматично.
```

53. Динамично управление на памет в Windows.

Windows предоставя механизми за виртуална памет и heap памет, върху които работят allocator-ите на приложението. C++ програмите могат да използват стандартните механизми или Windows API при нужда от по-ниско ниво на контрол. В обикновен C++ код се предпочитат RAII, контейнери и smart pointers.

54. Служебни структури в динамичния мениджмънт на паметта в Windows.

Heap manager-ът поддържа метаданни, чрез които следи заделените и свободните блокове памет. Точните вътрешни структури зависят от конкретната версия и реализация на Windows. Повреда на тези метаданни чрез out-of-bounds запис, use-after-free или double free може да доведе до срив и уязвимости.

55. Препълване на буфер в Windows и атаки, базирани на това.

Buffer overflow в Windows също е запис извън границите на заделен буфер и може да повреди памет или контролния поток. Съвременните защити включват DEP, ASLR, stack cookies и Control Flow Guard. Те затрудняват експлоатацията, но правилното решение остава предотвратяването на memory-safety грешки.